Assignment 2 — Kaggle Digit Recognizer

Oct 27, 2026 · @raya

Author: Raya Metchkarova · **Kaggle username:** rmetchkarovaieu2024-bit · **Public score:** 0.99539

1. Objective

The goal is to classify 28,000 handwritten 28×28 grayscale digits (0–9) from the Kaggle **Digit Recognizer** competition, scored by accuracy on the public leaderboard. The target is to beat the course reference of 0.97 (simple MLP) and reach 0.99 or more with a CNN.

Three PyTorch models are compared on the same hold-out validation set: an MLP baseline, a CNN with 2 convolutional blocks, and an improved CNN with BatchNorm, Dropout, data augmentation and a learning-rate scheduler. The best one produces the Kaggle submission.

All reusable code lives in the package `src/digit_recognizer/` (data, models, training, prediction, plots), installed with `pip install -e .` and imported by the notebook `notebooks/digit_recognizer.ipynb`.

2. Data preparation

Only Kaggle's `train.csv` (42,000 labelled images) and `test.csv` (28,000 unlabelled images) are used; no external digits and not the original MNIST test set.

1. **Loading** (`load_train_csv`, `load_test_csv`): each row holds 784 pixel values (0–255), reshaped to 28×28. The classes are roughly balanced (about 3,800–4,700 images per digit), so accuracy is a fair metric.
2. **Validation split** (`split_train_val`): a stratified 90/10 hold-out split with seed 42, giving 37,800 training and 4,200 validation images with the same digit proportions. The Kaggle test set is used only once, for the final predictions.
3. **Normalisation** (`DigitDataset`): pixels are divided by 255 (as `ToTensor()` does) and standardised with the mean and standard deviation of the **training split only** (as `Normalize(mean, std)` does with one channel). The same two numbers are reused for validation and test, so no information leaks from them.
4. **Tensor shape**: each image becomes `[1, 28, 28]`, so a batch is `[B, 1, 28, 28]` (batch, channels, height, width). Batch size is 128.

5. **Augmentation** (improved model only, training set only): `RandomAffine` with ±10° rotation, ±10% shift, 0.9–1.1 zoom and ±5° shear, applied before normalisation so empty borders stay black. Horizontal flips are not used, because a flipped 2 or 7 is not a valid digit. Validation and test images are never augmented.

3. Models and architectures

All three models take `[B, 1, 28, 28]` and return **logits** of shape `[B, 10]`; no softmax is applied inside the model because `CrossEntropyLoss` applies it internally. A pytest test checks this output shape for every model.

Model	Architecture	Parameters
MLP (baseline)	Flatten → Linear(784, 256) → ReLU → Linear(256, 128) → ReLU → Linear(128, 10)	235,146
CNN	2 blocks of Conv(3×3, padding 1) → ReLU → MaxPool(2), with 32 and 64 filters → Flatten → Linear(3136, 128) → ReLU → Linear(128, 10)	421,642
Improved CNN	2 blocks of [Conv → BatchNorm2d → ReLU] × 2 → MaxPool(2) → Dropout2d(0.1), with 32 and 64 filters → Flatten → Linear(3136, 256) → BatchNorm1d → ReLU → Dropout(0.4) → Linear(256, 10)	871,082

Shapes through the CNN: `(B, 1, 28, 28)` → first block `(B, 32, 14, 14)` → second block `(B, 64, 7, 7)` → flatten `(B, 3136)` → `(B, 10)`. Padding 1 with a 3×3 kernel keeps the height and width; each MaxPool(2) halves them.

The MLP sees the image as a flat vector of 784 numbers and ignores which pixels are neighbours. The convolutions reuse the same small filters at every position, so the CNN learns strokes and corners wherever they appear.

Improvement. The improved CNN keeps the same 2-block structure so the comparison with the CNN is fair, and combines several items from the course's list of improvements:

- **BatchNorm** after every convolution, to stabilise the feature maps and allow faster training.
- **Dropout** (Dropout2d after each block, Dropout 0.4 in the classifier), to reduce overfitting.
- **Data augmentation** with `RandomAffine` (section 2), so the model sees digits in many positions, sizes and slants.

- **CosineAnnealingLR** scheduler and label smoothing 0.1 (section 4).

4. Training and validation

Every model is trained with the same loop from the package (`fit` in `train.py`), which follows the course order for each batch: `zero_grad` → forward pass → loss on the logits → `backward` → `optimizer.step` . After every epoch the model is evaluated on the validation set in `model.eval()` mode with `torch.no_grad()` , so Dropout is off and BatchNorm uses its running statistics.

Setting	MLP	CNN	Improved CNN
Loss	Cross-entropy	Cross-entropy	Cross-entropy, label smoothing 0.1
Optimiser	Adam, lr 1e-3	Adam, lr 1e-3	Adam, lr 1e-3
LR scheduler	None	None	CosineAnnealingLR (1e-3 → 0 over 15 epochs)
Epochs	10	10	15
Augmentation	No	No	RandomAffine

- **Validation** is the stratified 10% hold-out set (4,200 images), identical for the three models. Validation loss is always plain cross-entropy, so the curves are comparable.
- **Checkpoint selection**: the weights of the epoch with the best validation accuracy are kept (`best_val_acc` , `best_epoch` in the history), a simple form of early stopping.
- **Reproducibility**: seed 42 for Python, NumPy and PyTorch, and a seeded generator for the shuffled training loader. The whole notebook ran in about 26 minutes on a 2-core CPU without a GPU (about 70 seconds per epoch for the improved CNN).

5. Results and Kaggle score

The improved CNN reaches **0.9964 validation accuracy** (15 errors out of 4,200), against 0.9898 for the CNN and 0.9793 for the MLP baseline, so each step halves the errors or better. Its Kaggle public score is **0.99539**. All numbers come from `results/model_comparison.csv` and `results/metrics.json` .

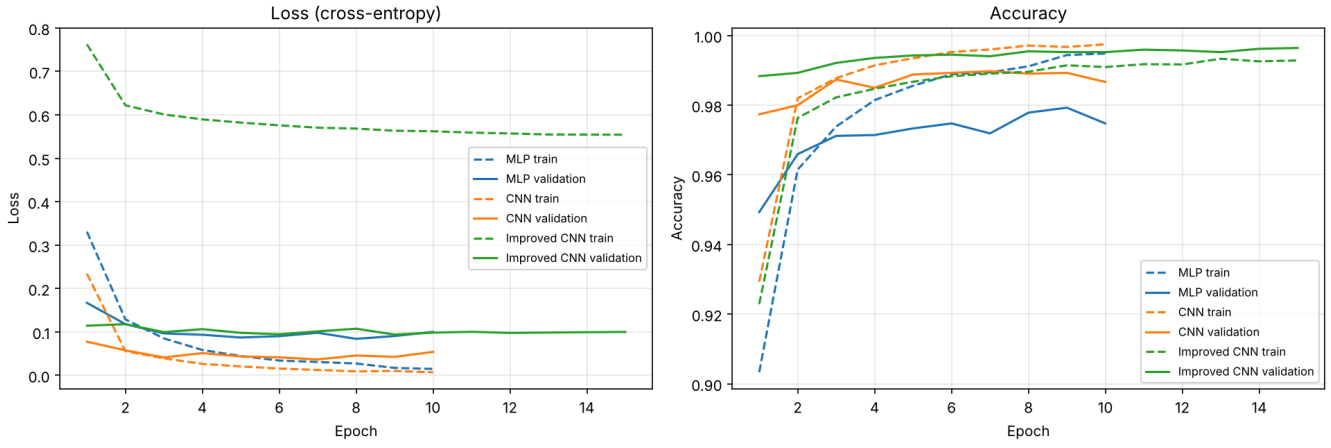
Model	Parameters	Best epoch	Final train accuracy	Best validation accuracy	Validation errors (of 4,200)
MLP	235,146	9 of 10	0.9948	0.9793	87

Model	Parameters	Best epoch	Final train accuracy	Best validation accuracy	Validation errors (of 4,200)
CNN	421,642	7 of 10	0.9975	0.9898	43
Improved CNN	871,082	15 of 15	0.9929	**0.9964**	**15**

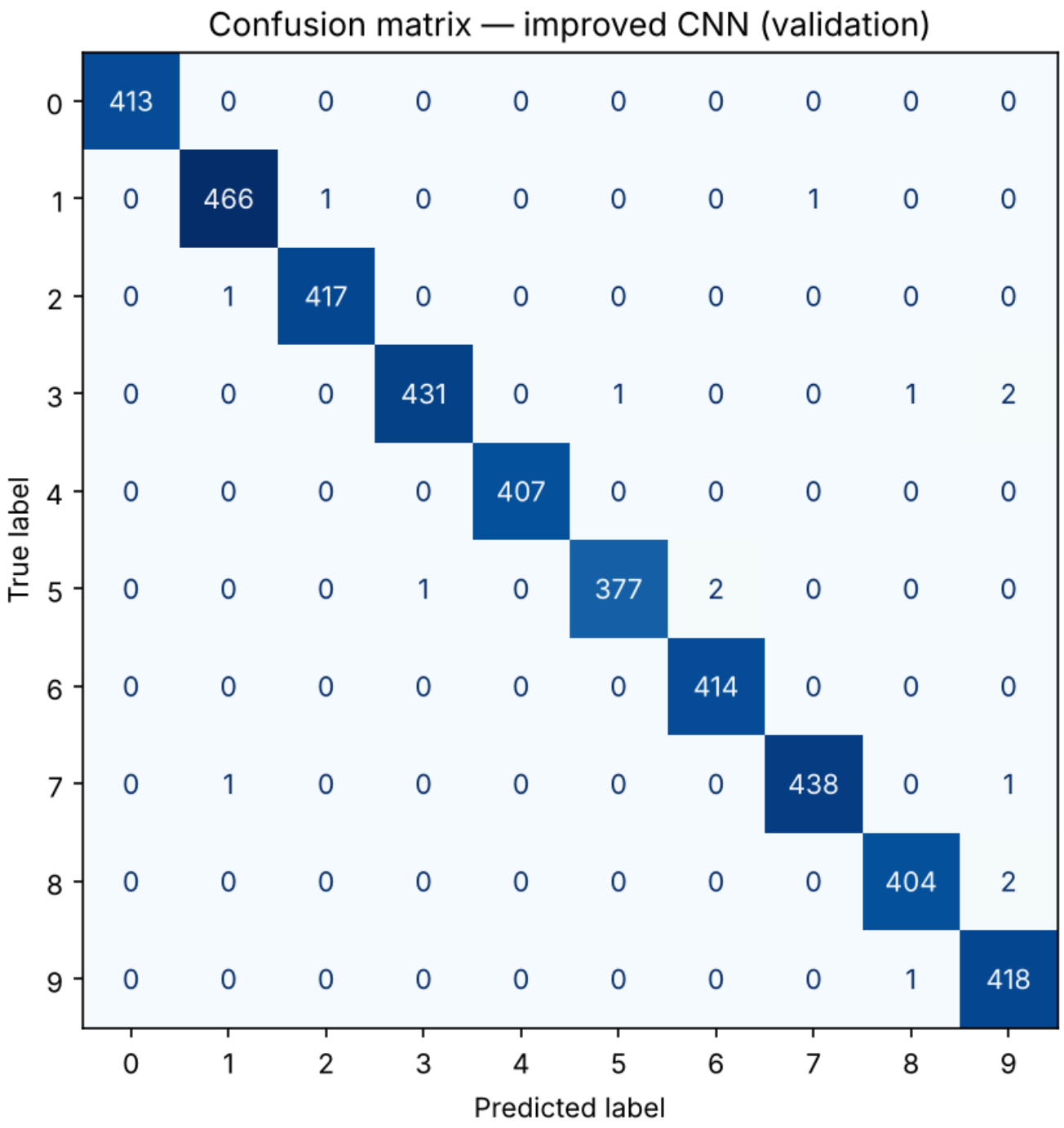
Kaggle submission. `results/submission.csv` (header `ImageId,Label`, 28,000 rows, ImageId 1–28,000, integer labels 0–9, no missing values, checked by `check_submission`) was produced by the improved CNN. Kaggle username: **rmetchkarovaieu2024-bit**. Public leaderboard score: **0.99539**.

The public score (0.99539) is close to the validation accuracy (0.9964). The small gap is expected: the validation set has only 4,200 images, so one image is worth 0.024 percentage points, and the public leaderboard uses a different subset of the test images. It confirms the hold-out validation was a reliable estimate and the model did not overfit to it.

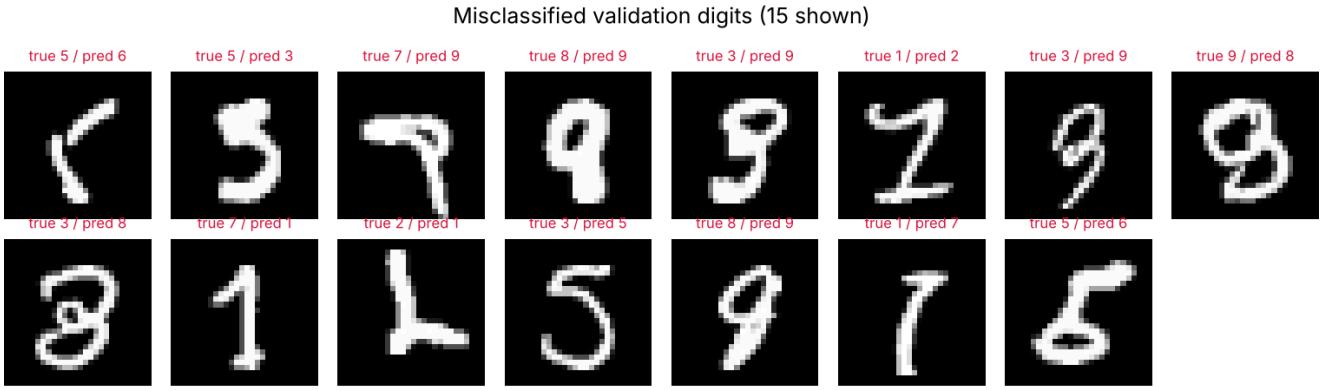
Learning curves. The MLP and the CNN reach 0.995–0.998 training accuracy while validation accuracy stalls (MLP around 0.975–0.979; CNN around 0.99, with validation loss rising after epoch 7): both overfit. For the improved CNN, validation accuracy stays above training accuracy for the whole run, because augmentation and dropout only act during training; its training loss stays near 0.55 because of label smoothing. Its validation accuracy keeps improving until the last epoch as the cosine schedule lowers the learning rate.



Confusion matrix (improved CNN, validation). The errors are spread out: no confusion happens more than twice (8→9, 5→6 and 3→9 twice each). Digits 0, 4 and 6 are classified perfectly; the lowest per-class accuracy is 0.9908, for digit 3.



Misclassified digits. Most of the 15 errors are digits a person could also misread: a 3 that looks like a 5, a 7 written like a 1, an 8 or 3 with an open loop read as a 9, a 5 with a closed bottom read as a 6. A few look mislabelled, so the remaining error is close to the noise in the labels.



6. Conclusions and limitations

Conclusions. Using the 2-D structure of the image matters most: a CNN with only 2 convolutional blocks halves the MLP's validation errors (87 → 43). Regularisation matters next: BatchNorm, Dropout, augmentation and the cosine schedule remove the overfitting seen in both baselines and cut the errors to 15 (0.9964 validation accuracy), well above the 0.99 target. This model produced the Kaggle submission.

Limitations.

- A single hold-out split and a single seed: the differences between models are measured on 4,200 images, so a difference of a few images is within noise. 5-fold cross-validation would give a more reliable estimate.
- The final model is trained on 90% of the labelled data; retraining on all 42,000 images with the chosen settings would likely help a little.
- The four improvements are applied together, so their individual contributions are not separated (no ablation study).
- Training on a laptop CPU limited the number of epochs and models. An ensemble of several CNNs or test-time augmentation could add a few tenths of a percent.

AI-use declaration

Claude (Anthropic) was used to help structure the package, draft the code and draft this report, starting from the course materials (Session 9 notebook, the CNN class notebook, the lecture notes and the packaging session). I ran all the code myself, checked every result reported here and can explain each part of the code.